

Iniziato lunedì, 26 gennaio 2026, 14:06

Stato Completato

Terminato lunedì, 26 gennaio 2026, 14:42

Tempo impiegato 36 min. 46 secondi

Domanda **1**

Completo

Punteggio max.: 2,00

Javascript è un linguaggio **debolmente tipato**. Cosa significa? Scegli la definizione corretta.

- ☒ a. Il motore di esecuzione può convertire automaticamente i dati da un tipo all'altro se necessario per portare a termine un'istruzione.
- ☐ b. I tipi dei dati elaborati dal codice sono noti solo nel momento in cui il programma è in esecuzione
- ☐ c. Il linguaggio permette di creare un oggetto senza passare dal costruttore di una classe
- ☐ d. Il linguaggio prevede il tipo **undefined** per gestire il casi in cui una variabile è vuota o una funzione non restituisce nulla

Domanda **2**

Completo

Punteggio max.: 4,67

Illustrare il pattern architetturale MVVM per come è stato presentato nel contesto di React.

in REACT abbiamo solo una pagina che si chiama App.js/jsx quindi altre pagina che navighiamo attraverso i browser sono componente quindi il react è un single page model
quindi quando apriamo una pagina che è generato con React, il motore di React solo vede URL che desideriamo di andare e il motore di React cambia componente in una sola pagina. significa la pagina App sempre aperto e per altre view che vediamo è solo componenti.
quindi questo modello di React si chiama Model-View-View-Model.

Domanda **3**

Completo

Punteggio max.: 2,00

Un componente React è definito come una funzione, detta **funzione di rendering**.

Cosa deve restituire tale funzione?

- ☐ a. uno stato definito con useState
- ☐ b. un XML
- ☐ c. Un'espressione JSX o TSX
- ☒ d. una Promise

Domanda **4**

Completo

Punteggio max.: 4,66

Descrivere il ciclo di vita delle @Entity nella Java Persistence API di Spring

nel server-side abbiamo JPA (Jakarta Persistence API) per comunicare e prendere le risorse da DB, questo lavora come ORM (Object-Relational-Model)

JPA lavora solo con Entity e Relation di un DB, quindi con Annotation dobbiamo specificare che modello di operazione vogliamo fare

@Entity è in realtà per accedere alle entità di una tabella di Data-Base e ha un'altra operazione @jpaRepository per far operazioni CRUD (create, read, update, delete).

abbiamo delle operazioni come "repo.save()" per salvare un record nella tabella

"repo.delete()" per rimuovere un record.

Domanda **5**

Completo

Punteggio max.: 2,00

Parliamo di come l'ambiente di esecuzione di Javascript gestisce gli eventi, in modo che sia possibile per il programmatore specificare il codice da eseguire in risposta agli eventi rilevanti per l'applicazione che sta scrivendo.

Quali delle seguenti affermazioni sono vere?

(risposta multipla)

- ☒ a. Gli eventi endogeni, ossia che sono conseguenza di richieste eseguite all'interno del codice (es.: fetch), sono bloccanti: l'ambiente di esecuzione resta in attesa di una risposta e non reagisce a nessun altro evento.
- ☒ b. I listener che un'applicazione associa agli eventi UI vengono invocati nello stesso ordine in cui gli eventi si verificano, senza eccezioni.
- ☒ c. L'ambiente di esecuzione gestisce gli eventi sequenzialmente in ordine di arrivo, con un unico thread.
- ☐ d. L'arrivo di un evento può interrompere la gestione dell'evento precedente, ed è necessario tenerne conto nello scrivere il codice

Domanda **6**

Completo

Punteggio max.: 2,00

Si consideri il seguente codice Javascript (le righe sono numerate per comodità):

```
1.     const elenco = ["Farfalla", "Gatto", "Airone"];  
2.     elenco[1] = "Coniglio";
```

L'istruzione alla riga 2 modifica l'array *elenco*, che era dichiarato usando la parola chiave **const**. È corretta o dà luogo ad un errore? Per quale ragione?

- ☐ a. Dà errore perché la dichiarazione *const* ammette che si modifichi l'oggetto da essa riferito invocandone un metodo, ma non che si vadano a modificarne direttamente i dati.
- ☒ b. È corretta perché l'istruzione alla riga 2 modifica il contenuto di una posizione già presente nell'array (la posizione 1); se aggiungesse o rimuovesse elementi invece darebbe errore.
- ☐ c. È corretta perché *elenco* contiene un riferimento all'oggetto array, che non viene modificato dall'istruzione alla riga 2. Se l'istruzione assegnasse ad *elenco* un nuovo array, darebbe errore.
- ☐ d. Dà errore perché la dichiarazione *const* implica che il valore assegnato ad *elenco* sia immutabile e dunque *elenco* non possa mai comparire sulla sinistra di un assegnamento.

Domanda 7

Completo

Punteggio max.: 2,00

Cosa sono le **Servlet** in un'applicazione server-side realizzata con Jakarta Web Profile?

- ☐ a. Sono il componente client-side delle applicazioni server-side realizzate con Jakarta Web Profile, e servono a definirne la user interface.
- ☐ b. Sono un elemento del middleware che si frappone fra il Web Container e l'applicazione vera e propria realizzata con Spring
- ☐ c. Sono il motore che gestisce il pattern Inversion of Control e si occupa di effettuare la dependency injection verso i Controller
- ☒ d. Sono oggetti che ricevono la richiesta HTTP e hanno la responsabilità di elaborare la risposta; come tali, sono il fondamento di un'applicazione server-side

Domanda 8

Completo

Punteggio max.: 2,00

Le applicazioni web server-side realizzate con Spring Web sono basate sul pattern Model-View-Controller. Per ciascuno dei seguenti elementi costitutivi di un'applicazione Spring Web stabilisci se fa parte del Model, della View, del Controller o di nessuno dei tre.

body della HTTP Response	View
@Entity	Model
@RestController	Controller
@Service	Model
body della HTTP Request	View

Domanda **9**

Completo

Punteggio max.: 4,67

Delineare le principali funzionalità di un Web Server

la web server in realta e un ogetto per accettare alle risorse.
in server-side abbiamo due componenti principali (HTTP Server) e (Web Server)
ci sono delle differenza fra questi, la HTTP Server e una macchina di esecuzione ch si chiama Host in cui ci sono applicazioni e in realta con operazione Route trovare ogni requesta è per quale applicazione
ma Web server e un oggetto per far comunicare le risorse

Domanda **10**

Completo

Punteggio max.: 2,00

Selezionare, fra i seguenti elementi, quelli che possiamo trovare in una URL.
(risposta multipla)

- ☒ a. protocollo
- ☐ b. header
- ☒ c. dominio
- ☒ d. query string
- ☐ e. method

Domanda **11**

Completo

Punteggio max.: 2,00

Una **differenza** fra una **richiesta HTTP** con metodo **GET** e una con metodo **POST** è che...

- ☐ a. una richiesta GET può utilizzare i cookie, una richiesta POST no
- ☐ b. una richiesta GET può contenere, nella URL, una *query string*, mentre una richiesta POST no
- ☐ c. una richiesta POST può partire solo da un form, mentre una richiesta GET può partire da un link o dal codice Javascript
- ☒ d. nelle richieste POST il *body* è usato per i dati da inviare, nelle richieste GET è assente o ignorato

Domanda **12**

Completo

Punteggio max.: 2,00

Quali delle seguenti affermazioni sui **@Service** nelle applicazioni web server-side realizzate con Spring/SpringBoot è **falsa**?

- ☐ a. Gli oggetti @Service sono *bean* gestiti dall'*Inversion of Control Container* che si occupa di inizializzarli e iniettarli all'interno di altri bean che ne facciano richiesta
- ☒ b. I metodi delle classi annotate con @Service devono avere nomi costruiti con una ben precisa logica che permette a Spring di interpretarne il senso
- ☐ c. Un @Service può utilizzare un altro @Service tramite *dependency injection*, per esempio richiedendolo come parametro nel proprio costruttore
- ☐ d. Per ogni classe con annotazione @Service viene creata di default una e una sola istanza.